exercise 10.

problem 1.

prove the statement by induction on the number of edges $m = |E|$.

Because all edge weights are distinct, the minimum-weight edge crossing any cut is unique. This lets us prove the stronger conclusion that it belongs to every MST, not just some MST.

If $m = |V| - 1$. then since G is connected, G is a tree. Therefore, G itself is the unique spanning tree, hence also the unique MST. For any cut $(S, V \setminus S)$, the minimum-weight edge crossing that cut is an edge of G, and since every edge of G belongs to the unique MST, that edge belongs to every MST of G. Thus the statement holds in the base case.

Assume that for every connected weighted graph with at most k edges, and for every cut in that graph, the minimum-weight edge crossing the cut belongs to every MST.

Let $G = (V, E, w)$ be a connected weighted graph with $k+1$ edges. Let $(S, V \setminus S)$ be any cut of G, and let e be the minimum-weight edge crossing this cut. We must show that e belongs to every MST of G.

Since $k+1 > |V| - 1$. the graph G contains at least one cycle. Choose any cycle C, and let h be the heaviest edge on C. Because edge weights are distinct, h is uniquely heaviest on that cycle.

we claim that h does not belong to any MST of G. Let T be any MST. Suppose, for contradiction, that $h \in T$. Removing h from T disconnects T into two components. But the rest of the cycle C still contains a path connecting the endpoints of h, so there is some other edge f on C that reconnects these two components. Since h is the heaviest edge on C, we have $w(f) < w(h)$. Then $T' = T - h + f$.

is also a spanning tree, and $w(T') = w(T) - w(h) + w(f) < w(T)$ which contradicts that T is an MST. Hence h belongs to no MST of G. Now remove h from G, and form

$$G' = (V, E \setminus \{h\}, w).$$

Then G' is still connected, because removing one edge from a cycle does not disconnect the graph. Also, G' has exactly k edges.

Moreover, the set of MSTs of G' is exactly the same as the set of MSTs of G, because h belongs to no MST of G.

Next, we show that $h \neq e$. If h does not cross the cut, then clearly $h \neq e$. If h does cross the cut, then because a cycle crosses any cut an even number of times, there must be another edge $g \neq h$ on the same cycle that also crosses the cut. Since e is the minimum-weight edge crossing the cut and weights are distinct, $w(e) < w(g)$.

Also, since h is the heaviest edge on the cycle. $w(g) < w(h)$

Therefore, $w(e) < w(h)$. So $e \neq h$.

Thus e is still present in G'. Since removing h does not remove any lighter crossing edge of the cut, e remains the minimum-weight edge crossing $(S, V \setminus S)$ in G'.

Now G' is a connected weighted graph with k edges, so by the inductive hypothesis, e belongs to every MST of G'. Since the MSTs of G' are exactly the MSTs of G, it follows that e belongs to every MST of G.

Therefore, the statement holds for graphs with $k+1$ edges. By induction, the statement is true for all connected weighted graphs. ※

Problem 2.

(a) Hierarchical feedforward cascades.

Algorithm: Topological Sort (and subsequently shortest / Longest Path on a DAG)

Justification: The phrase "strictly forward without loops" defines a Directed Acyclic Graph (DAG). Topological sorting determines the exact hierarchical sequence of information flow. If you need to find the specific cascade delay, you run a DAG Shortest Path algorithm on the sorted vertices.

Expected Time Complexity: $O(|V|+|E|)$

(b). Recurrent feedback circuits

Algorithm: Tarjan's Algorithm or Kosaraju's Algorithm (for finding Strongly connected Components - SCCs)

Justification: The phrase "circulates within a closed subset of regions" indicates a set of nodes where every node can reach every other node via directed paths. This is the exact definition of a Strongly Connected Component (SCC).

Expected Time Complexity: $O(|V|+|E|)$

(c). Fastest reactive pathways

Algorithm: Dijkstra's Algorithm

Justification: The goal is to "minimize delay" (find the shortest path) from a stimulus to an action. Since the problem states that weights encode delay and $w(u,v) > 0$ (no negative delays), Dijkstra's algorithm is the most efficient and appropriate method.

Expected Time Complexity: $O(|E| + |V| \log |V|)$ (using a Fibonacci heap) or $O((|V|+|E|) \log |V|)$ (using a standard binary heap).

Fundamental limitation of static graph algorithms for modeling real neural "thinking flows":

Limitation: Static graph algorithms cannot capture temporal dynamics and neuroplasticity. Real neurons have time-varying states (e.g. firing rates) and the "weights" (synaptic strengths) change dynamically based on activity (learning). Static graphs model continuous flow, whereas actual neural information propagates as discrete, non-linear spikes in a temporal sequence.